

which resources were generated, updated, or destroyed, allowing it to make the necessary adjustments.

Plan and apply workflow: Terraform includes a 'plan' command that allows you to preview changes before applying them, which improves predictability and safety. Before making the intended changes, users can review them.

Modularity and reusability: Terraform allows users to create reusable modules, which allow them to encapsulate configurations and promote code reuse across several projects and teams.

Installation

Before you begin working, ensure that Terraform is installed on your workstation; it may be installed on any operating system, such as Windows, macOS, Linux, or others. The process is simple and may be completed in a matter of minutes.

Visit Terraform's official downloads page at <https://developer.hashicorp.com/terraform/downloads>. Select the Terraform version you wish to install (usually the most recent stable version). Download the binaries for your operating system (Windows, macOS, or Linux, for example). Terraform is accessible on a variety of platforms. For Linux installation we use the following commands.

Import the HashiCorp GPG key:

This command retrieves the HashiCorp GPG key and puts it in `/usr/share/keyrings/hashicorp-archive-keyring.gpg`. This key is used to validate HashiCorp's software packages.

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg.
```

Insert the HashiCorp repository

here: The HashiCorp repository is added to the list of software sources using this command. The repository URL is determined by your Linux distribution.

`$(lsb_release -cs)` dynamically finds your distribution's code name, such as 'bionic' for Ubuntu.

```
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
```

Refresh the package list and

Terraform installation: This command refreshes your system's package list, ensuring that the newly installed HashiCorp repository is recognised by package management. Subsequently upgrading the package list, you may install Terraform using `apt install`. The package manager will download and install the most recent Terraform version from the HashiCorp repository.

```
sudo apt update && sudo apt install terraform.
```

We may use the `terraform cli` interface to verify the installation once it has been done successfully. You can verify the installation by running:

```
terraform --version
```

```
terraform {
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
      version = "~> 3.0.1"
    }
  }
}

provider "docker" {
  host = "unix:///var/run/docker.sock"
}

resource "docker_image" "nginx" {
  name = "nginx:latest"
  keep_locally = false
}

resource "docker_container" "nginx" {
  image = docker_image.nginx.image_id
  name = "tf-nginx"
  ports {
    internal = 80
    external = 8000
  }
}
```

Figure 1: Terraform configuration

Terraform configuration files

Terraform uses the HashiCorp Configuration Language, which is easy to understand, modify, update, and implement by the developer. Configuration files, which have the file extensions `.tf` and `.tf.json`, are used to specify Terraform defined infrastructure using a declarative approach. Configuration files allow you to specify your intended state by writing a configuration. These files are collections of resources that contain settings and values that describe the desired state of your infrastructure.

- **Configuration file (*.tf):** In this file, users define the provider and resources to be deployed, as well as the resource type and all resource-specific parameters.
- **Variable declaration file (variables.tf/variables.tf.json):** Here users declare the input variables required to provision resources.
- **Terraform state file (terraform.tfstate):** Terraform creates a state file just once. It saves information about users' managed infrastructure.

Now let's run and set up the nginx Docker container.

Running nginx Docker container using Terraform

Create a file name that ends with an extension `.tf <filename>.tf`. This configuration file is used to configure the cloud provider, network configuration, ports, image, keys, variables, etc, as shown in Figure 1.

Once the file is created and saved, run the command `terraform init` to apply the changes that install the mentioned plugins, and create a `lock` file on success. In case of any modification to the `.tf`

file we need to run the command again,